Système et Réseaux : Partiel 1

mars 2025 — Pierre Rousselin

Consignes et conseils :

- Les prototypes des appels système et de certaines fonctions sont rappelés en fin d'énoncé.
- Pour gagner du temps, sauf indication contraire, vous pourrez supposer que tous les appels système réussissent.
- Les inclusions de fichiers d'en-tête ne sont pas nécessaires non plus.
- Il est inutile de recopier l'énoncé. Lorsque vous devez compléter des programmes, il suffit d'indiquer clairement à quel endroit (numéro de ligne, corps de fonction...) vous ajoutez des instructions ou modifiez le programme.
- La clarté et la simplicité des solutions seront des éléments importants de l'évaluation.
- L'ordre des exercices correspond au plan du cours, pas à leurs difficultés.
- Le barème est donné à titre indicatif et est susceptible d'être modifié.

Exercice 1 : encore une *forkbomb* (6 points)

On considère le programme suivant :

```
1  #define N_DFT 5
2  #define P_DFT 2
3  int main(int argc, char *argv[])
4  {
5          int n = argc > 1 ? atoi(argv[1]) : N_DFT;
6          int p = argc > 2 ? atoi(argv[2]) : P_DFT;
7          while (0 < p && p < n) {
8                  if (fork() == 0)
9                          p--;
10                 n--;
11         }
12         printf("lol\n");
13         return 0;
14 }
```

On suppose que le programme compilé s'appelle `fb`.

1. (4 points) Dessiner l'arbre de création des processus correspondant à l'exécution de ce programme sans aucun argument. En particulier, combien de `lol` sont affichés ?

2. (2 points) Donner une formule de récurrence permettant de calculer, en fonction de `n` et `p` le nombre total de processus créés par la ligne de commande

```
$ ./fb n p
```

en supposant que l'utilisateur entre pour `n` et `p` des chaînes correspondant à des entiers positifs avec $\mathtt{p} \leq \mathtt{n}$. Justifier brièvement. En déduire une formule donnant explicitement ce nombre de processus.

--- * ---

Correction de l'exercice 1 :

1. Il y a 10 processus créés, autant de `lol` affichés. Voir la figure 1.

2. Soit $u_{n,p}$ ce nombre de processus. Si $n = p$ ou $p = 0$, alors le processus ne rentre pas dans la boucle donc ne fork pas et $u_{n,p} = 1$. Si $0 < p < n$, alors le processus fork puis décrémente n. Son enfant décrémente à la fois n et p, on a donc $u_{n,p} = u_{n-1,p} + u_{n-1,p-1}$ donc $u_{n,p} = \binom{n}{p}$.

--- * ---

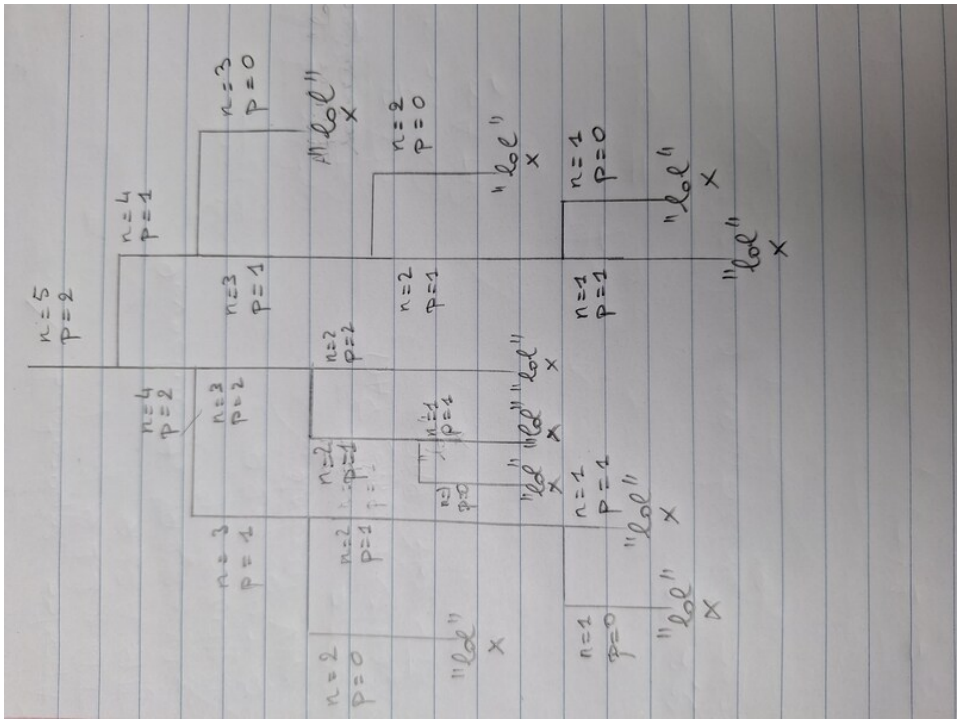

FIGURE 1 – Arborescence des processus de l'exercice 1

Exercice 2 : La course des processus (14 points)

Le programme `course` lance autant de processus qu'il a d'arguments dans sa ligne de commande et son premier enfant exécute le programme donné en premier argument, son deuxième enfant le programme donné en deuxième argument, etc.

Le processus qui gagne la course est le premier à se terminer

— sans être tué par un signal et

— avec un statut de sortie égal à 0 (succès).

Une fois le processus vainqueur de la course connu, les processus restants *qui ne sont pas déjà morts* sont tués par le processus parent.

On suppose que les processus ont des temps d'exécution suffisamment longs et différents pour que l'appel système qui attend leur mort annonce successivement leurs morts dans l'ordre.

Voici un exemple d'exécution où les programmes `a`, `b`, `c` et `d` font la course.

```
$ ./course ./a ./b ./c ./d
Processus d'indice 0, de PID 339597, de programme ./a lancé
Processus d'indice 1, de PID 339598, de programme ./b lancé
Processus d'indice 2, de PID 339599, de programme ./c lancé
Processus d'indice 3, de PID 339600, de programme ./d lancé
Processus d'indice 0, de PID 339597, de programme ./a fini avec statut de sortie 2
Processus d'indice 1, de PID 339598, de programme ./b tué par le signal 15
Processus d'indice 2, de PID 339599, de programme ./c est le vainqueur !
```

Ici, le premier enfant exécute le programme `./a`, il se termine en premier mais n'est pas le vainqueur car son statut de sortie n'est pas `0`. Le deuxième enfant exécute le programme `./b`, se termine juste après le premier enfant, mais n'est pas le vainqueur, car il a été tué par un signal. Le troisième enfant se termine normalement, avec un statut de sortie `0`, c'est donc le vainqueur. Pour finir, le quatrième enfant est tué par le processus parent car la course est terminée.

1. (2 points) Définir les deux fonctions suivantes :

```
/* retourner l'indice dans le tableau pids, de taille n, du premier élément
 * égal à p, s'il existe, -1 sinon. */
int index_of(pid_t p, const pid_t *pids, size_t n);

/* afficher l'indice, le PID et le programme exécuté, sans caractère de fin de
 * ligne final */
void afficher_proc(int index, pid_t pid, const char *prog);
```

La fonction `afficher_proc` doit être conforme aux débuts des affichages précédents, par exemple :

```
Processus d'indice 2, de PID 339599, de programme ./c
```

2. (2 points) Début de la fonction `main`, donner les instructions permettant :

— d'afficher un message d'erreur si le processus qui exécute `course` n'a pas au moins deux arguments et

— d'allouer sur le tas un tableau `pids` qui peut contenir autant de `PID` qu'il y a d'arguments dans la ligne de commande (dans l'exemple, il en contiendrait 4).

3. (4 points) Création des processus et exécution des programmes : donner les instructions permettant de créer autant de processus enfants qu'il y a d'arguments dans la ligne de commande, chaque enfant exécutant le programme donné en argument qui lui correspond. En cas d'échec de l'exécution du programme (par exemple s'il n'existe pas), afficher un message d'erreur utile et mettre fin au processus enfant avec un statut de sortie égal à 2. Le PID du premier enfant doit être stocké dans `pids[0]`, celui du deuxième enfant dans `pids[1]`, etc.

4. (4 points) Attente de la fin des enfants : donner les instructions permettant au processus parent d'attendre la fin de ses enfants, jusqu'au vainqueur éventuel. Lorsqu'un enfant se termine en étant tué par un signal ou bien avec un statut de sortie différent de 0, afficher, comme dans l'exemple ci-dessus, les précisions sur la mort de cette enfant. Pour les enfants qui sont terminés, mettre la case correspondante dans `pids` à 0.

5. (2 points) Enfin, donner les instructions permettant de mettre fin, avec un signal (par exemple `SIGTERM`), à tous les processus qui ne sont pas encore terminés et libérer toutes les ressources.

--- * ---

Correction de l'exercice 2 :

1.

```
void afficher_proc(int index, pid_t pid, const char *prog)
{
        printf("Processus d'indice %d, de PID %d, de programme %s",
                index, pid, prog);
}

int index_of(pid_t p, const pid_t *pids, size_t n)
{
        int i;
        for (i = 0; i < n; i++) {
                if (pids[i] == p)
                        return i;
        }
        return -1;
}
```

2. Ne pas pénaliser si le message d'erreur n'arrive pas dans `stderr`.

```
if (argc < 3) {
        fprintf(stderr, "Usage: course prog1 prog2...\n");
}
pid_t *pids = malloc(argc * sizeof(pid_t));
```

3. L'affichage n'était pas explicitement demandé dans l'énoncé, ne pas pénaliser s'il n'est pas présent. Idem pour le cas d'erreur de `fork`. Le cas d'erreur de `execlp`, beaucoup plus probable, doit par contre être traité.

```
int i;
for (i = 1; i < argc; i++) {
        switch ((pids[i - 1] = fork())) {
        case -1:
                perror("fork");
                break;
        case 0:
                execlp(argv[i], argv[i], NULL);
                perror("execlp: ");
                return 2;
        default:
                afficher_proc(i - 1, pids[i - 1], argv[i]); printf(" lancé\n");
        }
}
```

4. Bien sûr on peut aussi utiliser `WIFEXITED` et adopter la logique inverse.

```
pid_t p;
int status;
while ((p = wait(&status)) > 0) {
        int n = index_of(p, pids, argc - 1);
        pids[n] = 0;
        if (WIFSIGNALED(status)) {
                afficher_proc(n, p, argv[n + 1]);
                printf(" tué par le signal %d\n", WTERMSIG(status));
        } else {
                int exit_status = WEXITSTATUS(status);
                if (exit_status != 0) {
                        afficher_proc(n, p, argv[n + 1]);
                        printf(" fini avec statut de sortie %d\n", exit_status);
                } else {
                        afficher_proc(n, p, argv[n + 1]);
                        printf(" est le vainqueur !\n");
                        break;
                }
        }
}
```

5. Le plus important est la partie `kill`. Enlever des points si le test de `pid` n'est pas fait : en toute généralité, un système pourrait avoir réattribué les PID des processus morts. La partie `wait` n'est pas obligatoire (après tout, on les a tué...), mais est plus propre (les signaux peuvent mettre un peu de temps à être reçu).

```
for (i = 0; i < argc - 1; i++) {
        if (pids[i] != 0) {
                kill(pids[i], SIGTERM);
        }
}
while (wait(NULL) != -1)
        ;
free(pids);
```

--- * ---

Exercice 3 : *threads* pour calculs en parallèle (10 points)

Dans cet exercice, le *thread* initial commence par allouer dynamiquement sur le tas et remplir de valeurs aléatoires entre 0 et `MAX - 1` deux tableaux, de même taille, `tab1` et `tab2`.

Il s'agit ensuite de faire calculer deux nouveaux tableaux à deux autres *threads*, l'un contenant la somme élément par élément de `tab1` et `tab2` ; l'autre le produit élément par élément. Le squelette du programme, à compléter, est donné ci-dessous.

```c
void *calc_somme(void *tabs);
void *calc_prod(void *tabs);
#define MAX 20
/* retourne l'adresse d'un tableau alloué dynamiquement sur le tas,
   de taille n, rempli par des valeurs pseudo-aléatoires entre 0 et MAX - 1. */
double *remplir_tab(size_t n);
void afficher_tab(double *tab, size_t n);
/* ZONE D'AJOUT 1 */
int main()
{
        size_t n;
        printf("taille du tableau ? ");
        scanf("%lu", &n);
        double *tab1 = remplir_tab(n);
        double *tab2 = remplir_tab(n);
        printf("tableau 1 : "); afficher_tab(tab1, n);
        printf("tableau 2 : "); afficher_tab(tab2, n);
        /* ZONE D'AJOUT 2 */
        /* ZONE D'AJOUT 3 */
        return 0;
}
double *remplir_tab(size_t n) { /* À DÉFINIR 1 */ }
void afficher_tab(double *tab, size_t n)
{
        size_t i;
        for (i = 0; i < n; i++)
                printf("%g ", tab[i]);
        printf("\n");
}
void *calc_somme(void *tabs) { /* À DÉFINIR 2 */ }
void *calc_prod(void *tabs) { /* SUPPOSÉE DÉFINIE */ }
```

Voici un exemple d'exécution pour vous montrer ce qui est attendu :

```
taille du tableau ? 5
tableau 1 : 3 6 17 15 13
tableau 2 : 15 6 12 9 1
somme : 18 12 29 24 14
produit : 45 36 204 135 13
```

1. (2 points) Définir la fonction `remplir_tab`, en suivant la spécification informelle donnée à sa déclaration.
2. (2 points) Définir un type structuré `struct deux_tabs`, permettant aux fonctions `calc_somme` et `calc_produit` d'avoir les informations nécessaires pour pouvoir calculer leurs tableaux respectifs.
3. (2 points) Définir la fonction `calc_somme` qui attend en argument un pointeur vers une variable de type `struct deux_tabs`, alloue sur le tas un nouveau tableau de même taille que les tableaux

initiaux, le remplit avec la somme élément par élément des tableaux donnés dans argument et enfin retourne l'adresse de ce tableau.

Vous omettrez la définition de la fonction `calc_prod`*, très similaire et la supposerez déjà écrite.*

4. (4 points) Écrire les instructions dans `main` permettant de lancer les deux *threads*, puis d'attendre qu'ils se terminent, d'afficher leurs résultats, et enfin de libérer la mémoire allouée par chacun d'eux, puis par `remplir_tab`.

--- * ---

Correction de l'exercice 3 :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#define MAX 20
void *calc_somme(void *tabs);
void *calc_prod(void *tabs);

double *remplir_tab(size_t n);
void afficher_tab(double *tab, size_t n);
struct deux_tab {
        double *tab1;
        double *tab2;
        size_t n;
};
int main()
{
        size_t n;
        printf("taille du tableau ? ");
        scanf("%lu", &n);
        double *tab1 = remplir_tab(n);
        double *tab2 = remplir_tab(n);
        printf("tableau 1 : "); afficher_tab(tab1, n);
        printf("tableau 2 : "); afficher_tab(tab2, n);

        struct deux_tab dt = { .tab1 = tab1, .tab2 = tab2, .n = n };
        pthread_t t_s, t_p;
        pthread_create(&t_s, NULL, calc_somme, &dt);
        pthread_create(&t_p, NULL, calc_prod, &dt);

        void *res;
        pthread_join(t_s, &res);
        printf("somme : "); afficher_tab(res, n);
        free(res);
        pthread_join(t_p, &res);
        printf("produit : "); afficher_tab(res, n);
        free(res);
        free(tab1);
        free(tab2);
        return 0;
}
double *remplir_tab(size_t n)
{
```

```
        double *res = malloc(n * sizeof(double));
        size_t i;
        for (i = 0; i < n; i++)
                res[i] = rand() % MAX;
        return res;
}
void afficher_tab(double *tab, size_t n)
{
        size_t i;
        for (i = 0; i < n; i++)
                printf("%g ", tab[i]);
        printf("\n");
}

void *calc_somme(void *tabs)
{
        struct deux_tab *dt = tabs;
        double *res = malloc(dt->n * sizeof(double));
        size_t i;
        for (i = 0; i < dt->n; i++)
                res[i] = dt->tab1[i] + dt->tab2[i];
        return res;
}

void *calc_prod(void *tabs)
{
        struct deux_tab *dt = tabs;
        double *res = malloc(dt->n * sizeof(double));
        size_t i;
        for (i = 0; i < dt->n; i++)
                res[i] = dt->tab1[i] * dt->tab2[i];
        return res;
}
```

--- * ---

Exercice 4 : Repas, micro-ondes et table (14 points)

Sur le campus de Villetaneuse, le restaurant du personnel est fermé depuis 1 an. Dans la salle café du LIPN, les enseignants réchauffent leurs plats au four à micro-ondes puis vont manger.

Voici un code non synchronisé qui modélise cette situation.

```
/* inclusions de fichiers d'en-tête... */
#define NUM_CHAISES 11
#define NUM_ENSEIGNANTS 40
void *enseignant(void *arg);
void *micro_ondes(void *arg);
void travailler(void);
void chantonner(void);
void prendre_plat(void);
void dormir(void);
void manger(void);
void rechauffer(void);
/* Zone 1 : autres variables globales à ajouter ici */
int main(void)
```

```
{
        int i;
        /* Zone 2 : initialisation des variables ajoutées ici */
        /* création des threads */
        /* attente des threads */
        /* libération des ressources */
        return 0;
}
void *enseignant(void *arg)
{
        for (;;) {
                travailler();
                /* attendre qu'un micro-ondes soit disponible */
                chantonner();
                /* attendre que le plat soit réchauffé */
                prendre_plat();
                manger();
                dormir();
        }
        return arg;
}
void *micro_ondes(void *arg)
{
        for (;;) {
                /* attendre qu'il y ait un plat à réchauffer */
                rechauffer();
                printf("le plat est réchauffé\n");
                /* attendre que le plat soit pris */
                printf("le plat est pris\n");
        }
        return arg;
}
```

Les fonctions `chantonner`, `rechauffer`, `dormir`, ... sont supposées écrites et sans effet de bord ; vous ne ferez aucune hypothèse sur leurs durées. Le lancement de `N` *threads* `enseignant` et d'1 *thread* micro-ondes est supposé fait correctement dans la partie `/* création des threads */`. De même, la partie `/* attente des threads */` est supposée correcte.

1. (6 points) En utilisant des sémaphores, synchroniser les fonctions `enseignant` et `micro_ondes` de manière de telle sorte que :

- le four micro-ondes n'est utilisé que par un enseignant à la fois ;
- un enseignant doit attendre que le four micro-ondes soit disponible avant de pouvoir l'utiliser ;
- l'enseignant chantonne pendant que le micro-ondes réchauffe son plat ;
- lorsque le micro-ondes a fini de réchauffer le plat, il continue à être allumé jusqu'à ce que l'enseignant prenne son plat ;
- une fois que le plat est pris par l'enseignant, le four micro-ondes est disponible pour d'autres enseignants.

Vous indiquerez quels sémaphores sont à ajouter dans la `/* Zone 1 */`, comment ils sont initialisés dans la `/* Zone 2 */`, et écrirez complètement les versions synchronisées des fonctions `enseignant` et `micro_ondes`. Enfin, vous indiquerez comment, dans la partie `/* libération des ressources */`, libérer les ressources associées à un des sémaphores.

2. En fait, il y a dans la salle café deux endroits où manger : la table, autour de laquelle il y a

`NUM_CHAISES` chaises et le canapé qui, pour simplifier, peut accueillir autant d'enseignants que nécessaire.

Les enseignants préfèrent toujours manger autour de la table. S'il n'y a pas de place ils vont s'asseoir sur le canapé pour manger. On définit deux constantes symboliques de valeurs distinctes `LIBRE` et `OCCUPE` ainsi qu'un tableau global : `int chaises[NUM_CHAISES]`. Dans la fonction `main` (`/* Zone 2 */`), toutes les cases de ce tableau sont mises initialement à la valeur `LIBRE`.

a) (1 point) Définir la fonction suivante :

```
/* retourner l'indice d'une chaise libre s'il y en a une ou -1 sinon */
int chaise_libre(void)
```

b) (4 points) On remplace la fonction `manger` par deux fonctions : `manger_table` et `manger_canape`. En utilisant un *mutex*, faire en sorte qu'un enseignant prêt à manger s'assoie autour de la table pour manger si une chaise est libre, sinon mange assis sur le canapé. Bien sûr, après avoir fini de manger, un enseignant qui a la chance d'avoir été assis autour de la table libère sa chaise. Donner les modifications à apporter dans les zones 1 et 2, dans la fonction `enseignant`, et dire comment libérer les ressources associées à un *mutex* à la fin du programme.

3. (3 points) Malheureusement, le canapé, trop sale, a été enlevé. Il n'y a maintenant que la table pour manger et un enseignant doit donc attendre qu'une place se libère à table avant de pouvoir manger. En utilisant sémaphores et/ou *mutex* et/ou variables globales, modifier le programme (zones 1 et 2, fonction `enseignant`) pour tenir compte de ce changement.

--- * ---

Correction de l'exercice 4 :

1. J'ai ajouté 4 sémaphores globaux, il y a certainement d'autres solutions, mais celle-ci colle bien au problème.

```
sem_t plat_a_rechauffer, plat_rechauffe, micro_dispo, plat_pris;
```

Le seul sémaphore initialisé à 1 est `micro_dispo`, les autres sont à 0. Dans `main`, avant la création des *threads* ajouter

```
sem_init(&plat_a_rechauffer, 0, 0);
sem_init(&plat_rechauffe, 0, 0);
sem_init(&micro_dispo, 0, 1);
sem_init(&plat_pris, 0, 0);
```

Les fonctions `enseignant` et `micro_ondes` sont modifiées comme suit :

```
void *enseignant(void *arg)
{
        for (;;) {
                int chaise;
                travailler();
                sem_wait(&micro_dispo);
                sem_post(&plat_a_rechauffer);
                chantonner();
                sem_wait(&plat_rechauffe);
                prendre_plat();
                sem_post(&plat_pris);
                dormir();
        }
        return arg;
}

void *micro_ondes(void *arg)
{
```

```
        for (;;) {
                sem_wait(&plat_a_rechauffer);
                rechauffer();
                sem_post(&plat_rechauffe);
                printf("%lu: le plat est réchauffé\n", pthread_self());
                sem_wait(&plat_pris);
                printf("%lu: le plat est pris\n", pthread_self());
                sem_post(&micro_dispo);
        }
        return arg;
}
```

Enfin, pour libérer un sémaphore (bon ici, ce n'était pas tellement intéressant, c'était juste pour tester cette compétence...) dans la zone de libération des ressources, donc après la fin (qui n'arrive jamais) des *threads* :

```
sem_destroy(&micro_dispo); /* idem pour les autres */
```

2. a)

```
int chaise_libre(void)
{
        int i;
        for (i = 0; i < NUM_CHAISES; i++)
                if (chaises[i] == LIBRE)
                        return i;
        return -1;
}
```

b) Modifier la fonction `enseignant` de la façon suivante. Attention au *deadlock* dans le cas où il n'y a pas de chaise libre...

```
sem_post(&plat_pris);

pthread_mutex_lock(&mut_chaises);
if ((chaise = chaise_libre()) == -1) {
        pthread_mutex_unlock(&mut_chaises);
        manger_sur_canape();
} else {
        chaises[chaise] = OCCUPE;
        pthread_mutex_unlock(&mut_chaises);
        manger_sur_table();
        pthread_mutex_lock(&mut_chaises);
        chaises[chaise] = LIBRE;
        pthread_mutex_unlock(&mut_chaises);
}
dormir();
```

3. Si on ne cherche pas à assigner une chaise à chaque enseignant (ce que l'énoncé n'imposait pas explicitement), la solution la plus simple est d'utiliser un sémaphore initialisé à `NUM_CHAISES`. Donc on déclare un nouveau sémaphore global :

```
sem_t chaises_libres;
```

initialisé avant la création des *threads* dans `main` :

```
sem_init(&chaises_libres, 0, NUM_CHAISES);
```

et la fonction `enseignant` est modifiée de la façon suivante :

```
sem_post(&plat_pris);
sem_wait(&chaises_libres);
manger_sur_table();
sem_post(&chaises_libres);
```

```
dormir();
```

Si l'on veut en plus attribuer une chaise à chaque enseignant, cela devient :

```
sem_post(&plat_pris);

sem_wait(&chaises_libres);
pthread_mutex_lock(&mut_chaises);
int chaise = chaise_libre();
chaises[chaise] = OCCUPE;
pthread_mutex_unlock(&mut_chaises);
manger_sur_table();
pthread_mutex_lock(&mut_chaises);
chaises[chaise] = LIBRE;
pthread_mutex_unlock(&mut_chaises);
sem_post(&chaises_libres);
dormir();
```

L'utilisation du *mutex* est obligatoire pour éviter, par exemple, que deux enseignants prennent la même chaise.

--- * ---

Prototypes des fonctions et appels systèmes à utiliser

Certains arguments sont mis à `NULL` ou `0` car nous n'avons pas vu leurs rôles en cours.

```
void *malloc(size_t n);
void free(void *p);
pid_t fork();
pid_t wait(int *wstatus);
/* Les macros pour utiliser le statut renseigné par wait sont
 * WIFEXITED, WIFSIGNALED, WEXITSTATUS et WTERMSIG */
int execlp(const char *file, const char *arg, ..., NULL);
int kill(pid_t pid, int sig);
int pthread_create(pthread_t *thread, NULL,
                       void *(*start_routine) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
int pthread_mutex_init(pthread_mutex_t *mutex, NULL);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
int sem_init(sem_t *sem, 0, unsigned int valeur);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);
```